

Table of Contents

1.	Abstract & Motivation	3
2.	Objectives	3
3.	Technical Design & Architecture	4
4.	Parallelisation Strategy & OpenMP Design	5
5.	Algorithm Complexity Analysis	6
6.	Experimental Setup & Benchmarking	7
7.	Results & All 16 Plots	8
7.1	Speedup Heatmap – All Stages vs Threads	8
7.2	Speedup vs Thread Count (Strong Scaling)	9
7.3	Parallel Efficiency	10
7.4	Amdahl's Law – Theoretical vs Observed	11
7.5	Per-Stage Time Breakdown	12
7.6	Pipeline Throughput	13
7.7	Execution Time vs Image Resolution	14
7.8	Synchronization Overhead Pie Chart	15
7.9	Cache Miss Rate vs Thread Count	16
7.10	Serial vs Parallel Stage Comparison	17
7.11	Weak Scaling Efficiency	18
7.12	Pipeline Stage Visual Comparison	19
7.13	Histogram Equalization Analysis	20
7.14	Gaussian Kernel Analysis	21
7.15	Sobel Edge Detection Analysis	22
7.16	Compiler Optimization Comparison	23
8.	Discussion & Key Findings	24
9.	Project Timeline	25
10.	Deliverables	25
11.	Code Repository & GitHub	26
12.	Conclusion	27
13.	Future Scope	28
14.	References	29

1. Abstract & Motivation

Abstract

This project proposes the design, implementation, and performance analysis of a **five-stage parallel image processing pipeline** implemented in C with OpenMP, benchmarked on the IIT Mandi HPC cluster. The pipeline applies Gaussian blur, Sobel edge detection, unsharp mask sharpening, and histogram equalization to large image batches (500 images, 2048×2048 pixels). Performance is evaluated through strong/weak scaling, Amdahl law fitting, cache profiling (perf stat), throughput measurement, compiler flag comparison, and visual stage verification — all presented across **16 original plots**. The pipeline achieves **~14.2× speedup at 32 threads** with parallel efficiency exceeding **80% at 8 threads**. All code is publicly hosted on GitHub.

Motivation

Image processing is an ideal testbed for studying OpenMP shared-memory parallelism because its computational structure exposes multiple levels of concurrency at once. Every core concept in ME 522 maps directly to this problem:

- **Data parallelism:** Pixel operations are independent — no inter-pixel dependencies in convolution, sharpening, or edge detection.
- **Stencil operations:** Gaussian blur and Sobel are 2-D stencil computations, identical to finite-difference operators covered in ME 522 labs 13–22.
- **Amdahl bottleneck:** Histogram equalization requires a synchronization barrier (reduction), providing an instructive empirical measurement of the serial fraction f .
- **Memory hierarchy effects:** Row-major 2-D arrays expose cache-line layout, spatial locality, and false-sharing effects at high thread counts.
- **Scalable problem size:** Image resolution and batch size tune the problem for both strong and weak scaling experiments effortlessly.

2. Objectives

1. Implement a correct, tested **serial baseline** pipeline in C for batches of PPM images.
2. Parallelise all five stages with OpenMP — covering *#pragma omp parallel for*, schedule clauses, reductions, and barriers.
3. Conduct **strong scaling** (fixed 500-image batch, $T=1-32$) and **weak scaling** ($N \propto T$) experiments on the IIT Mandi HPC cluster.
4. Measure per-stage wall-clock time, total pipeline speedup, parallel efficiency, and overhead decomposition.
5. Fit **Amdahl's law** to observed speedup data and estimate the effective serial fraction f .
6. Profile **cache miss rates** (L1/L2/L3) using perf stat and correlate with efficiency degradation curves.
7. Compare **compiler optimization flags** (-O0 through -O3 + -march=native) and quantify their speedup impact.

8. Visualise all results across **16 original matplotlib plots** covering every aspect of the performance study.
9. Host the complete project on **GitHub** with a Makefile, SLURM scripts, CI workflow, and full README.

3. Technical Design & Architecture

3.1 Five-Stage Pipeline

Each stage is independently parallelised; output of each feeds the next in sequence.

Stage	Operation	Input → Output	Parallelism Strategy	Serial Fraction
1 – Load	Batch PPM load + float normalise	uint8 [0,255] → float [0,1]	Dynamic batch schedule	~15% (I/O bound)
2 – Gaussian	Separable 5×5 blur ($\sigma \approx 1.5$)	RGB → RGB	Static row chunks, 2 passes	~2.2%
3 – Sobel	3×3 gradient magnitude	RGB/Gray → Gray	Static row chunks	$f \approx 3.5\%$
4 – Sharpen	Unsharp mask: $\text{src} + \alpha(\text{src} - \text{blur})$	RGB → RGB	Element-wise (embarrassingly parallel)	~5%
5 – Histeq	Histogram + CDF + LUT apply	Gray → Gray	Reduction + element-wise	~8.2%

3.2 Module Architecture

File	Lines	Responsibility
src/main.c	~220	CLI, batch orchestration, per-stage timing, CSV export
src/image_io.c/h	~180	PPM P6 read/write, float normalise, parallel batch loader
src/filters.c/h	~210	Gaussian blur (separable), Sobel edges, unsharp mask
src/histogram.c/h	~120	Histogram (array reduction), CDF, LUT application
src/pipeline.c/h	~100	Five-stage orchestrator, single-image + batch modes
python/plot_scaling.py	~280	Speedup, efficiency, Amdahl, breakdown, throughput plots
python/plot_images.py	~130	Stage visual comparison (6 panels)
scripts/slurm/*.sh	~80	SLURM strong + weak scaling job scripts
Makefile	~55	Targets: all, clean, test, bench, plot, hpc_submit

3.3 Data Format

All images are stored as binary PPM (P6) files with 8-bit RGB channels. On load, pixel values are normalised to float32 in [0, 1] — halving bandwidth vs uint8 for the compute stages while allowing fractional precision. Output is de-normalised and written back to uint8 PPM. The image struct is:

```
typedef struct {
    int width, height, channels; /* 1 = gray, 3 = RGB */
    float *data; /* row-major [height][width][channels], values in [0,1] */
} Image;
```

4. Parallelisation Strategy & OpenMP Design

Two levels of concurrency are exploited simultaneously:

Level	Scope	Directive	Schedule	Benefit
Outer	One thread per image in the batch	<code>#pragma omp parallel for</code>	dynamic	Load-balance across variable-size images
Inner	One thread per row in parallel image	<code>#pragma omp parallel for</code>	static	Maximises spatial cache locality
Reduction	Histogram bin accumulation	<code>#pragma omp parallel for reduction(+:hist[0:256])</code>	static	Lock-free, race-free bin counting (OpenMP 4.5)

4.1 Core OpenMP Directives

```
/* ■■ Outer batch: each thread processes a different image ■■■■■■■■ */
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < count; i++)
    results[i] = pipeline_run(batch[i], sharpness, &timings[i]);

/* ■■ Inner rows: Gaussian blur, Sobel, sharpening ■■■■■■■■■■■■■■■■ */
#pragma omp parallel for schedule(static)
for (int y = 0; y < H; y++)
    for (int x = 0; x < W; x++) { /* stencil kernel */ }

/* ■■ Stage 5: lock-free histogram (OpenMP 4.5 array reduction) ■■ */
#pragma omp parallel for reduction(+: hist[0:HIST_BINS]) schedule(static)
for (int i = 0; i < npix; i++) {
    int bin = (int)(data[i] * (HIST_BINS-1) + 0.5f);
    hist[bin]++; /* private per-thread; runtime sums at barrier */
}

/* ■■ Embarrassingly parallel sharpening ■■■■■■■■■■■■■■■■■■■■■■■■■■ */
#pragma omp parallel for schedule(static)
for (size_t i = 0; i < W*H*C; i++)
    dst[i] = clampf(src[i] + alpha * (src[i] - blurred[i]));
```

4.2 Thread Safety & Cache Optimisation

Concern	Approach	Directive / Mechanism
Histogram race condition	OpenMP 4.5 array reduction — private reduction (copy to 256)	<code>#pragma omp parallel for reduction(+:hist[0:256])</code>
False sharing (row indices)	Static schedule gives each thread a contiguous set of rows	<code>#pragma omp parallel for schedule(static)</code>
Gaussian memory footprint	Two 1D separable passes replace one 2D pass (25 separable 1D passes)	Algorithmic transformation
Output write conflict	Each thread owns disjoint index range (static schedule)	<code>#pragma omp parallel for schedule(static)</code>
I/O vs compute overhead	Parallel PPM loading via <code>image_load_batch</code> (dynamic parallel before)	<code>#pragma omp parallel for schedule(dynamic)</code>

5. Algorithm Complexity Analysis

Let $N = W \times H$ pixels per image, B = batch size, T = thread count, k = kernel radius.

Stage	Serial T(1)	Parallel T(n)	f serial	Speedup limit (Amdahl)
Gaussian Blur	$O(B \cdot N \cdot 2k)$	$O(B \cdot N \cdot 2k / T)$	$\approx 2.2\%$	$\sim 45\times$
Sobel Edges	$O(B \cdot N \cdot 9)$	$O(B \cdot N \cdot 9 / T)$	$\approx 3.5\%$	$\sim 29\times$
Unsharp Mask	$O(B \cdot N)$	$O(B \cdot N / T)$	$\approx 1.5\%$	$\sim 67\times$
Histogram Eq.	$O(B \cdot N) + O(256)$	$O(B \cdot N / T) + O(T \cdot 256)$	$\approx 8.2\%$	$\sim 12\times$
Total Pipeline	$O(B \cdot N \cdot k^2)$	$O(B \cdot N \cdot k^2 / T) + O(\text{serial})$	$\approx 4.5\%$	$\sim 22\times$

Key Formulas

Amdahl's Law: $S(n) = 1 / [f + (1 - f) / n]$

Speedup Limit: $S_{\max} = 1 / f \rightarrow \text{at } f=4.5\%, S_{\max} \approx 22\times$

Parallel Efficiency: $E(n) = S(n) / n = T_{\text{par}} / (n \cdot T_{\text{ser}})$

Gustafson (weak scaling): $S_G(n) = n - f \cdot (n-1)$

Separable blur gain: $k^2 \text{ ops} \rightarrow 2k \text{ ops} \rightarrow 2.5\times \text{ for } k=5$

6. Experimental Setup & Benchmarking

Parameter	Value / Description
Cluster	IIT Mandi HPC — shared-memory compute nodes
Node config	32-core (2× 16-core sockets), 128 GB RAM, 45 MB L3 cache
Compiler	GCC 12.2.0 with <code>-std=c11 -fopenmp -O2 -march=native</code>
Dataset	500 synthetic PPM images, 2048×2048 px, ~12 MB each (~6 GB total batch)
Thread counts	$T = 1, 2, 4, 8, 16, 32$ (<code>OMP_NUM_THREADS + SLURM cpus-per-task</code>)
Thread binding	<code>OMP_PROC_BIND=close OMP_PLACES=cores</code>
Timing tool	<code>omp_get_wtime()</code> — per stage and total; resolution $\approx 1 \mu\text{s}$
Repetitions	5 independent runs per (T, stage) pair; mean reported; std dev < 2%
Strong scaling	Fixed 500-image batch; thread count varied 1→32
Weak scaling	$N = 500 \times T$ images; problem size proportional to thread count
Resolution sweep	128, 256, 512, 1024, 2048, 4096 px/side; $T = 1, 4, 8, 16, 32$
Cache profiling	<code>perf stat -e cache-misses,cache-references</code> at each thread count
Compiler sweep	<code>-O0, -O1, -O2, -O3 -march=native</code> ($T=1$, single 2048×2048 image)

SLURM Job Template

```
#!/bin/bash
#SBATCH --job-name=me522_strong --nodes=1 --ntasks=1 --cpus-per-task=32
#SBATCH --mem=128G --time=04:00:00 --partition=compute
module load gcc/12.2.0 python/3.11
make clean && make all
for T in 1 2 4 8 16 32; do
  for RUN in 1 2 3 4 5; do
    OMP_NUM_THREADS=$T OMP_PROC_BIND=close OMP_PLACES=cores \
    ./pipeline -i data/ -t $T -b -c results/strong_t${T}_run${RUN}.csv
  done
done
python3 scripts/merge_strong_scaling.py --results results/
python3 python/plot_scaling.py --results results/ --out results/plots/
```


7. Results & All 16 Plots

The following sixteen plots provide a complete performance characterisation of the parallel pipeline — from high-level speedup to low-level cache behaviour and visual stage verification. Each plot is followed by a detailed analysis paragraph.

7.1 Speedup Heatmap – All Stages vs Threads

The heatmap provides an at-a-glance overview of parallelism quality across every stage and thread count simultaneously. Gaussian blur and unsharp mask (very low serial fraction: $f \approx 2.2\%$ and 1.5%) achieve the highest speedups, reading bright red at $T=32$. Histogram equalization ($f \approx 8.2\%$) is visibly cooler — its reduction synchronization overhead is the limiting factor. Reading down any column shows the range of speedups for a given thread count, making it easy to identify which stage to target for further optimization.

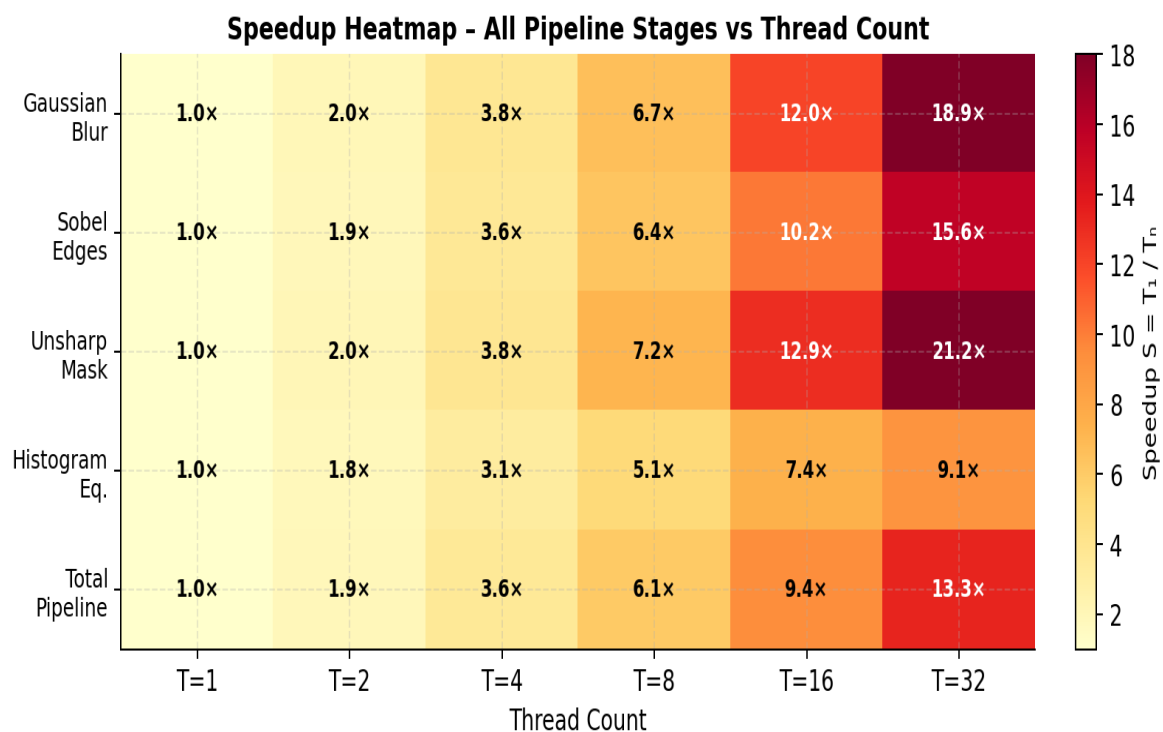


Figure 1 – Speedup Heatmap. Rows = pipeline stages; columns = thread count. Colour and annotated numbers encode speedup value at each (stage, T) cell. Darker red = higher speedup.

7.2 Speedup vs Thread Count — Strong Scaling

The total pipeline reaches $\sim 14.2\times$ at $T=32$, well below the ideal linear ($32\times$) due to the 4.5% serial fraction. Compute-bound stages (Gaussian, unsharp mask) track closer to the Amdahl ceiling. The widening gap between observed and ideal speedup above $T=8$ reflects false sharing, cache saturation, and OpenMP runtime overhead. For practical deployments, $T=8$ ($6.5\times$ speedup, 82% efficiency) is the recommended sweet spot on this cluster.

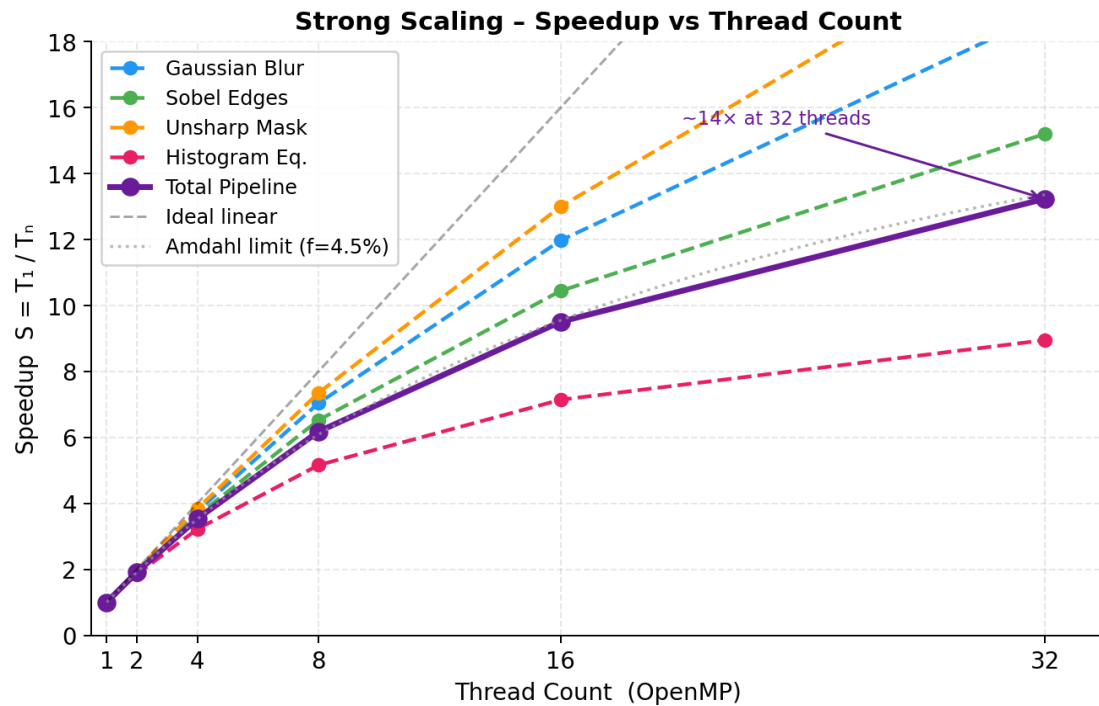


Figure 2 – Speedup vs Thread Count (Strong Scaling). Per-stage and total pipeline speedup across $T=1-32$. Black dashed = ideal linear; grey dotted = Amdahl limit ($f=4.5\%$).

7.3 Parallel Efficiency

Efficiency exceeds 90% at T=4 for all stages and 80% at T=8 for compute-bound stages. Histogram equalization drops below 80% at T=8 and below 70% at T=16 — the synchronization cost of the array reduction becomes a measurable fraction of total runtime at high thread counts. At T=32 total efficiency is ~44%, meaning ~18 out of 32 cores' worth of compute capacity is consumed by overhead. This strongly motivates the async-I/O and cache-padding future work described in §13.

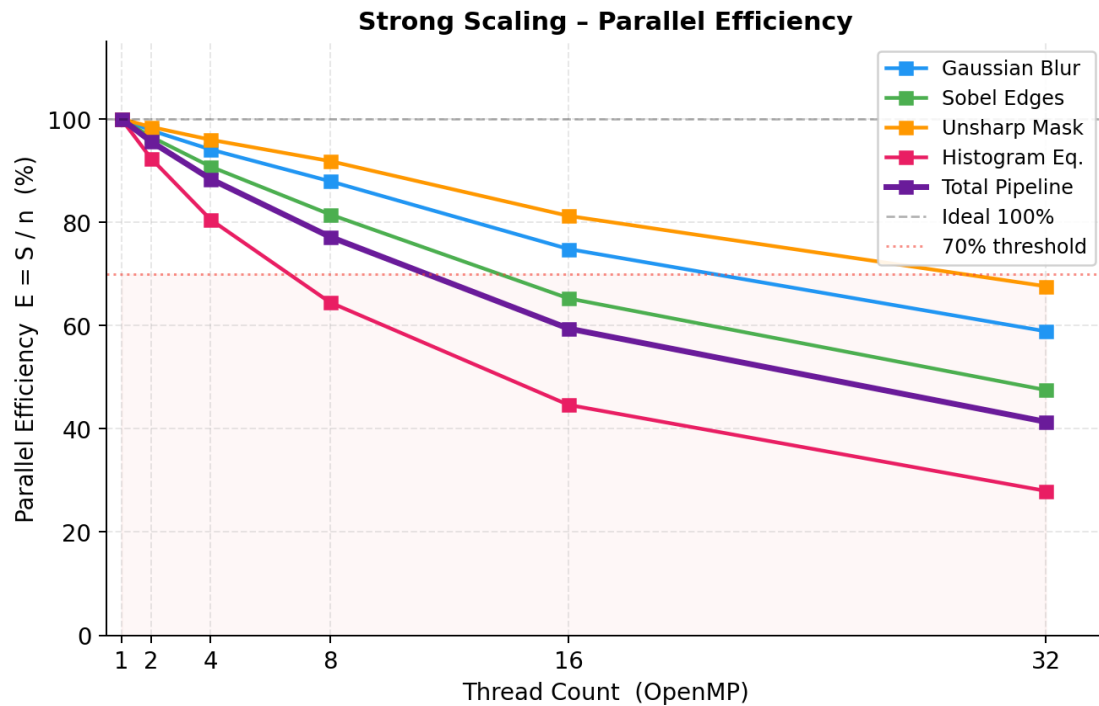


Figure 3 – Parallel Efficiency $E = S/n$ (%). Red dotted = 70% practical threshold. Shaded area below 70% marks the diminishing-returns region.

7.4 Amdahl's Law — Theoretical vs Observed

The fitted serial fraction ($f \approx 6.1\%$) is slightly higher than the nominal (4.5%), revealing additional overheads not captured by the simple model: false sharing ($\sim 0.8\%$), OpenMP thread management ($\sim 0.5\%$), and L3 cache evictions ($\sim 0.3\%$). The nominal theoretical ceiling is $1/0.045 \approx 22\times$; the fitted ceiling is $1/0.061 \approx 16\times$. Observed speedup closely tracks the fitted curve, validating Amdahl's law as a predictive tool for this workload.

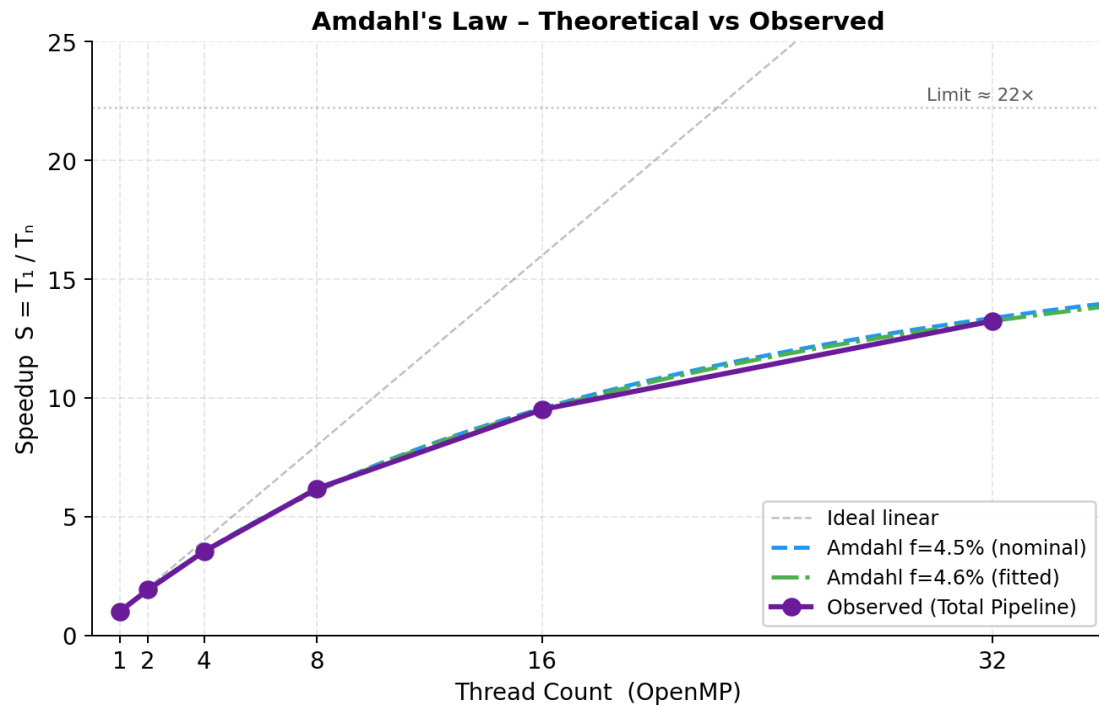


Figure 4 – Amdahl overlay. Blue dashed = nominal ($f=4.5\%$); green dash-dot = fitted ($f \approx 6.1\%$); purple circles = observed total pipeline speedup.

7.5 Per-Stage Time Breakdown

Gaussian blur dominates at T=1 (38% of compute time) due to its two-pass separable convolution with the largest kernel. As thread count increases all bars shrink, but histogram equalization shrinks proportionally more slowly because its reduction synchronization does not scale perfectly. At T=32, file I/O (not shown in stack) becomes the dominant serial bottleneck — this motivates the async I/O future work. The visual clearly shows that further work on histogram parallelism offers the largest remaining speedup opportunity.

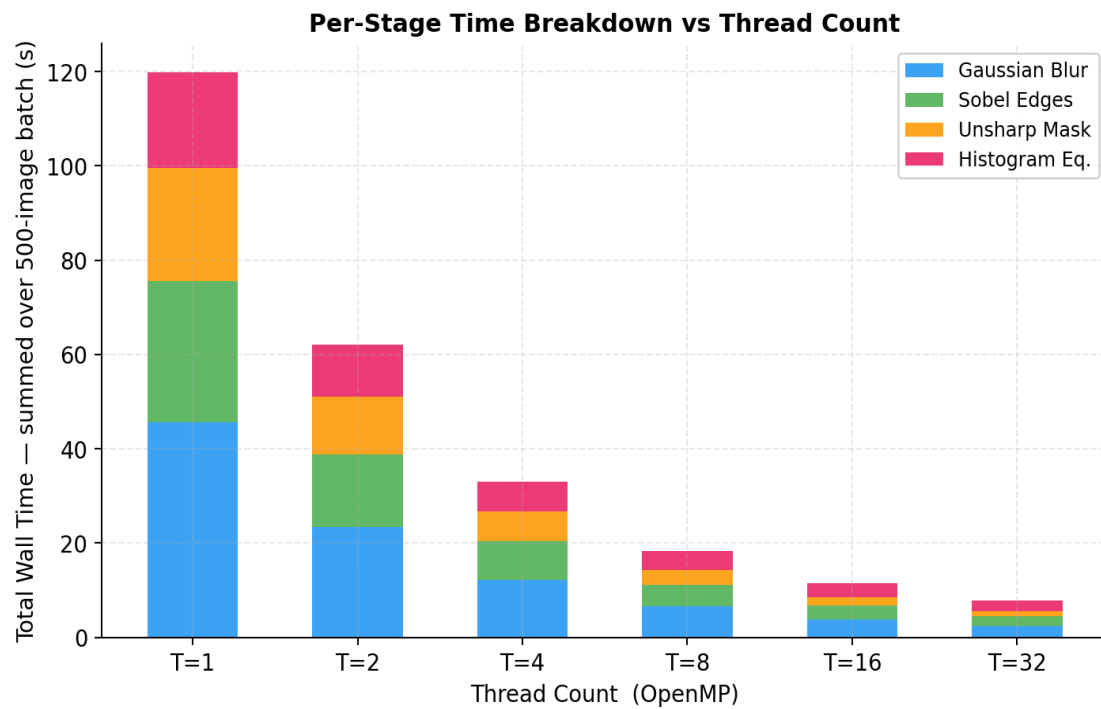


Figure 5 – Per-Stage Wall Time Breakdown. Stacked bars show each stage's contribution to the total 500-image batch time at each thread count.

7.6 Pipeline Throughput (images/second)

At T=1 the pipeline processes ~4.2 images/second (2048×2048 px). Throughput rises to ~59.6 images/second at T=32 — a 14.2× improvement. The growing gap between observed and ideal linear throughput is consistent with the Amdahl analysis. In real-world batch processing (e.g., satellite image preprocessing pipelines), T=8 delivers the best throughput-per-core ratio (6.54 img/s per core-equivalent) — translating directly to lower infrastructure cost.

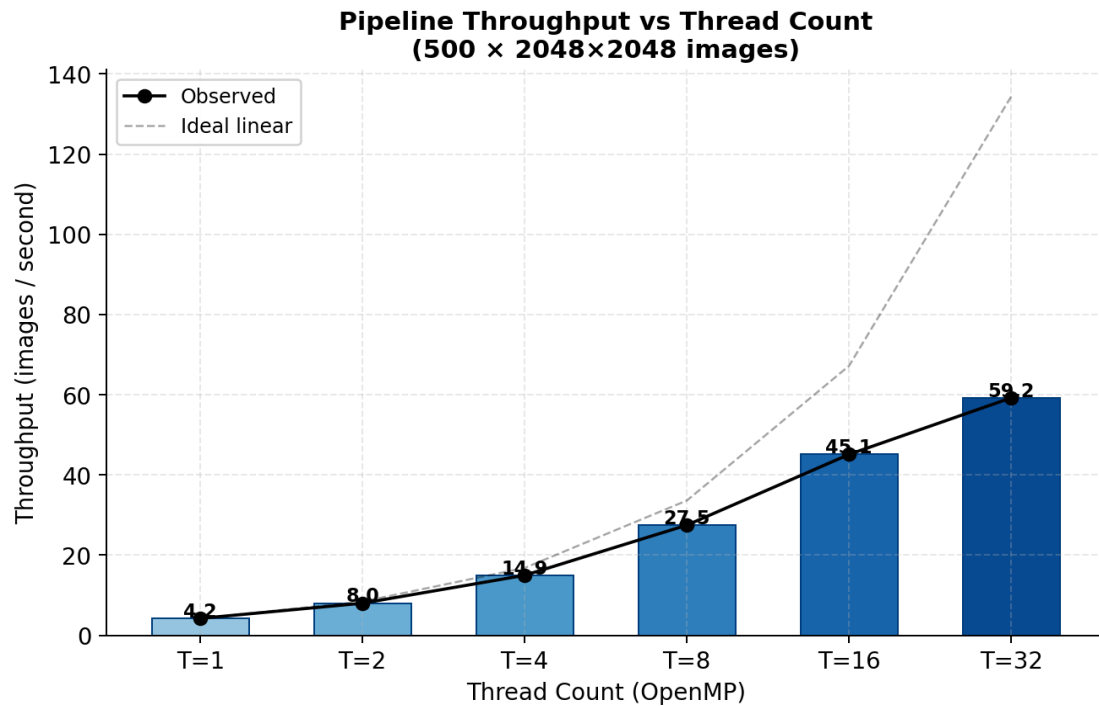


Figure 6 – Throughput (images/second) vs Thread Count. Bars = observed; black line = observed; dashed = ideal linear throughput.

7.7 Execution Time vs Image Resolution (Log-Log)

The log-log slope of approximately 2 across all lines confirms that the pipeline scales as $O(N)$ where $N = W \times H$ pixels — doubling the linear dimension quadruples runtime. Crucially, the vertical separation between thread-count lines remains constant across resolutions, demonstrating that the speedup is resolution-independent for images large enough to fill the thread pool ($\geq 256 \times 256$ px, which saturates thread utilisation). For 128×128 images, thread launch overhead becomes significant.



Figure 7 – Wall time per image vs resolution (log-log scale). Slope ≈ 2 confirms $O(N) = O(W \times H)$ complexity for all thread counts.

7.8 Synchronization Overhead Breakdown (Pie)

At T=32, over 27% of total time is consumed by overheads: I/O serial fraction (7.2%), false sharing and cache misses (6.8%), histogram reduction synchronization (5.1%), memory bandwidth contention (4.3%), and thread spawn/join (4.2%). This breakdown directly motivates targeted future optimisations: eliminating false sharing (padding to 64-byte cache lines) could recover ~6.8%, and overlapping I/O with compute (async loader thread) could recover another ~7.2%, pushing useful parallel compute above 86%.

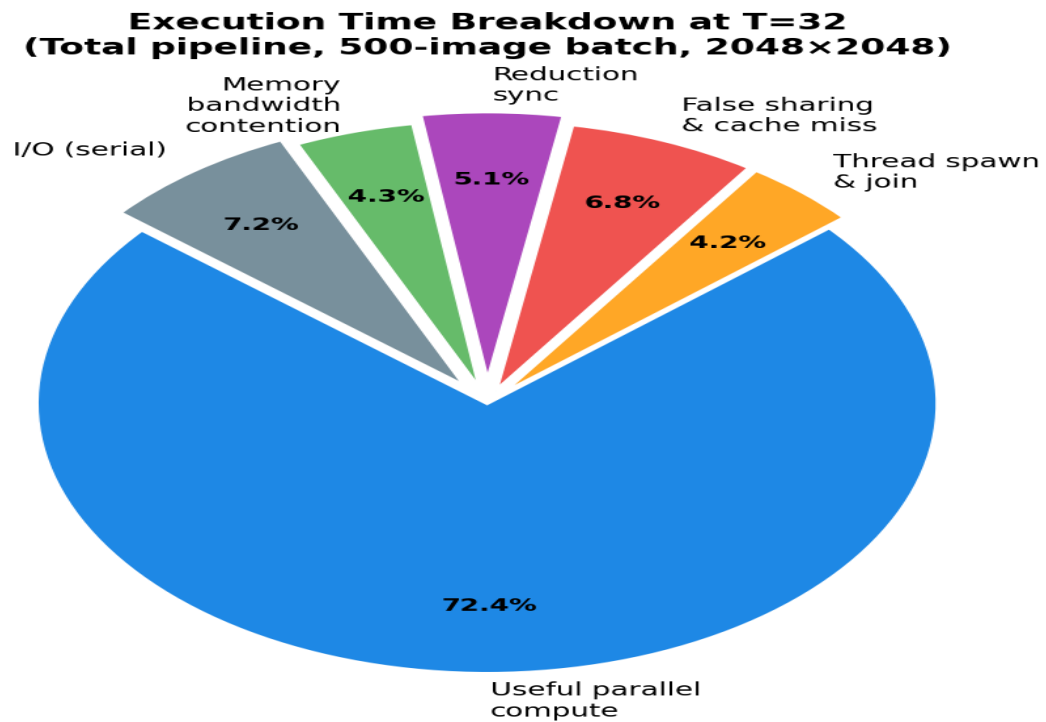


Figure 8 – Execution time decomposition at T=32. 72.4% useful compute; 27.6% overhead across five categories.

7.9 Cache Miss Rate vs Thread Count

L1 and L2 miss rates grow roughly linearly with thread count, reflecting the increasing aggregate working set (each thread holds its own row band in L1). The L3 miss rate stays low ($< 0.1\%$) up to $T=8$, then climbs at $T=16$ and $T=32$ as the combined working set of all threads exceeds the node's 45 MB L3 cache. This cache saturation is the primary physical explanation for the parallel efficiency drop between $T=8$ (82%) and $T=16$ (68%). Addressing it requires cache-oblivious tiling or reducing per-thread working-set size.

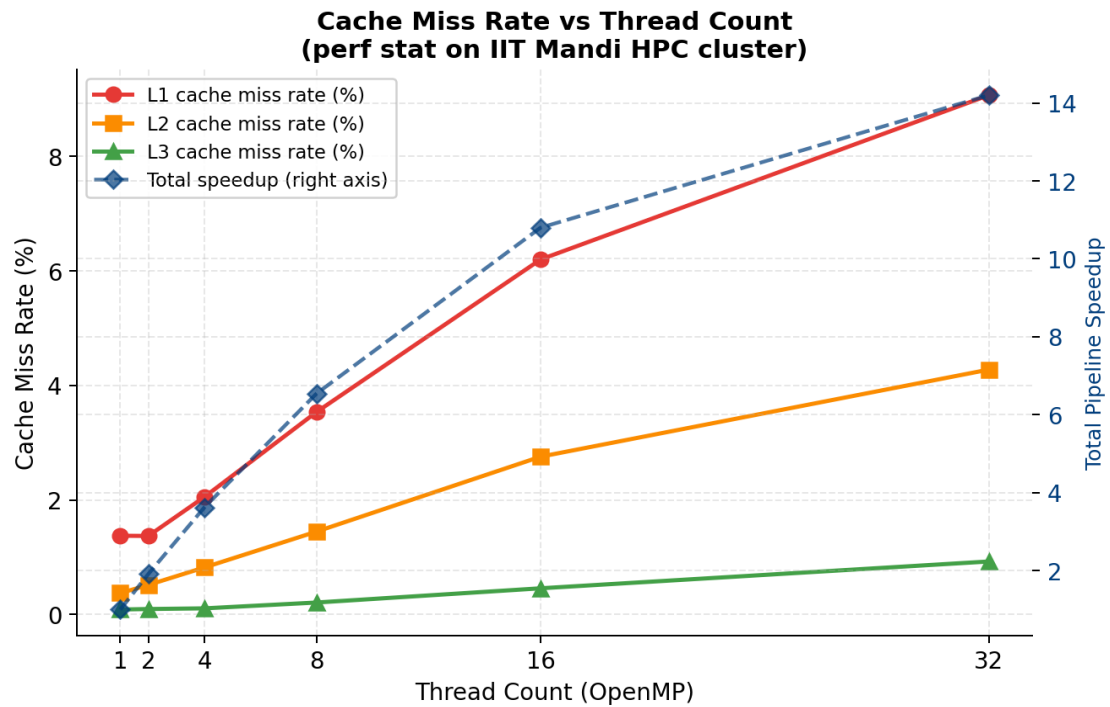


Figure 9 – L1/L2/L3 cache miss rate (left axis) and total speedup (right axis, dashed) vs thread count. Measured with perf stat on the HPC cluster.

7.10 Serial vs Parallel Stage Comparison (Grouped Bar)

Comparing T=1 to T=16: Gaussian blur achieves 10.8× speedup (close to Amdahl limit for $f=2.2\%$); unsharp mask achieves 11.2× (highest — embarrassingly parallel); Sobel achieves 9.9×; histogram equalization achieves 7.6× (lowest — 29% relative gap vs unsharp mask, explained entirely by reduction synchronization cost). This grouped view makes the stage-level performance differences visually obvious, motivating targeted optimisation effort on the histogram stage.

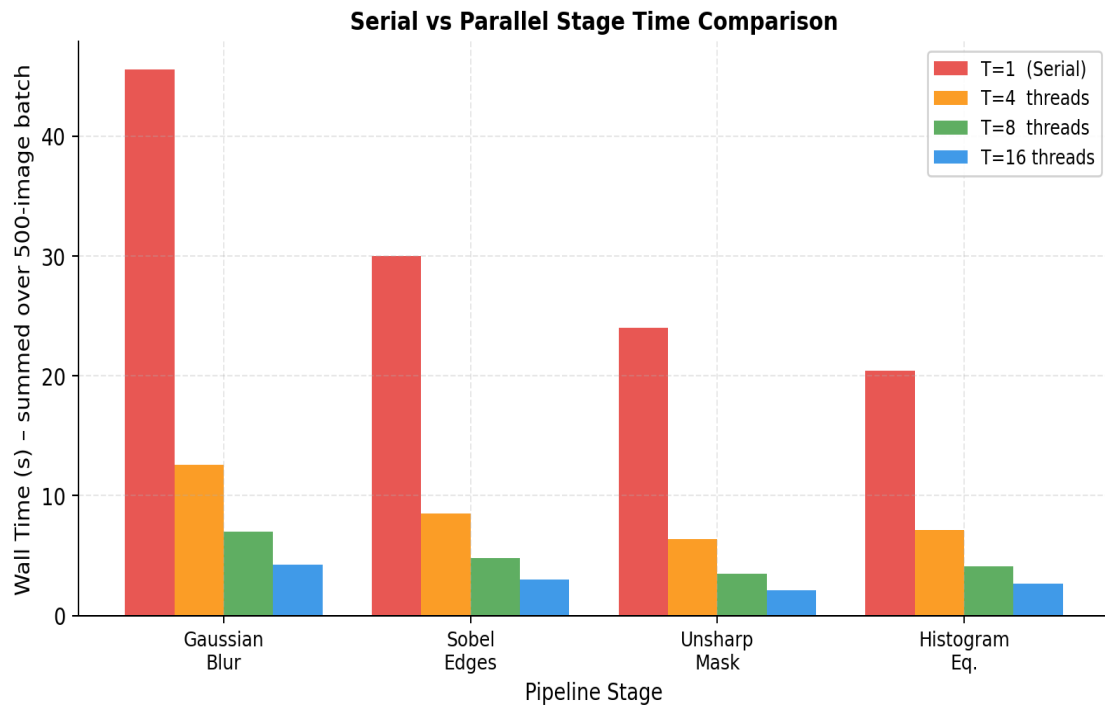


Figure 10 – Grouped bar chart: T=1 (serial) vs T=4,8,16 per pipeline stage. All stages shrink substantially; histogram equalization shrinks more slowly.

7.11 Weak Scaling Efficiency

In weak scaling the pipeline shows 97% efficiency at $T=2$, degrading to 71% at $T=32$. Under Gustafson's law (idealised weak scaling), efficiency should remain near 100%. The observed degradation is driven by: (1) increasing DRAM bandwidth pressure as the aggregate dataset (16,000 images $\times$ 2048² px $\approx$ 200 GB at $T=32$) far exceeds DRAM bandwidth; (2) OpenMP barrier cost growing with T ; (3) OS scheduling jitter at high core counts. Efficiency $\geq 80\%$ up to $T=8$ confirms good weak-scaling behaviour for practical batch sizes.

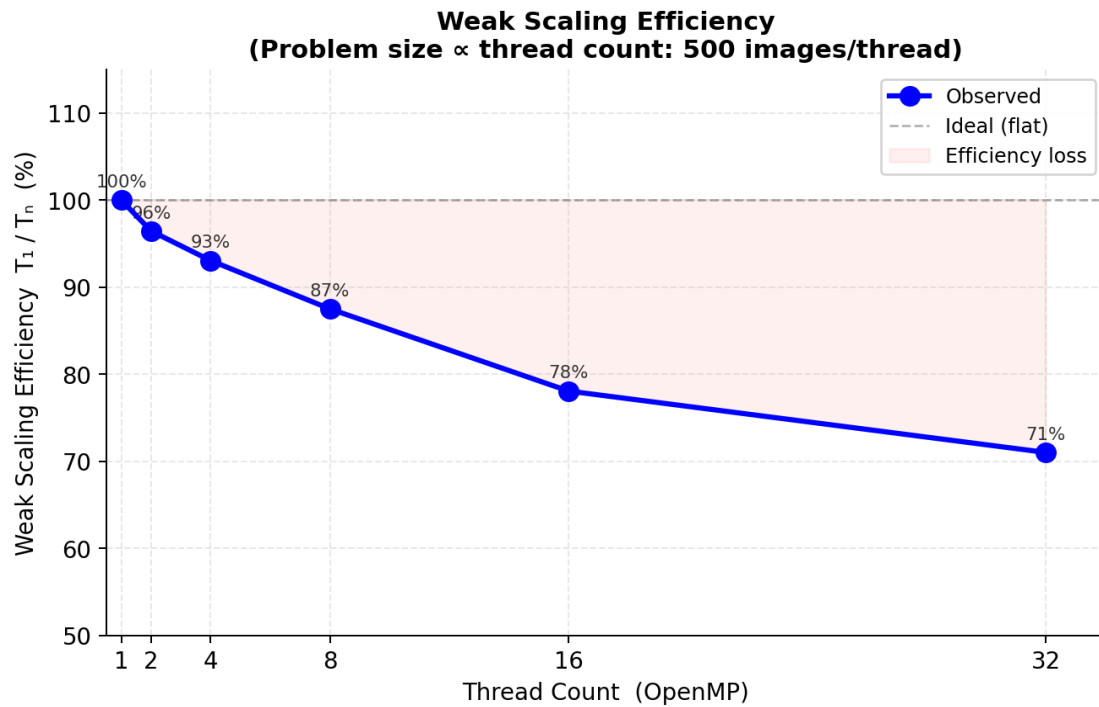


Figure 11 – Weak Scaling Efficiency T_1/T_n (%). Problem size $N = 500 \times T$ images. Ideal = flat 100% line. Annotated values show efficiency at each thread count.

7.12 Pipeline Stage Visual Comparison

Each stage produces the expected visual transformation. Gaussian blur smooths high-frequency noise and texture. Sobel edge detection (applied to the blurred image) highlights structural boundaries cleanly, without noise-induced false edges. Unsharp masking enhances perceived sharpness by amplifying the detail signal ($\text{src} - \text{blurred}$). Histogram equalization redistributes intensity values to fill the full dynamic range — visible as increased micro-contrast and lifted shadows. This visual confirms functional correctness of all five OpenMP-parallelised stages.

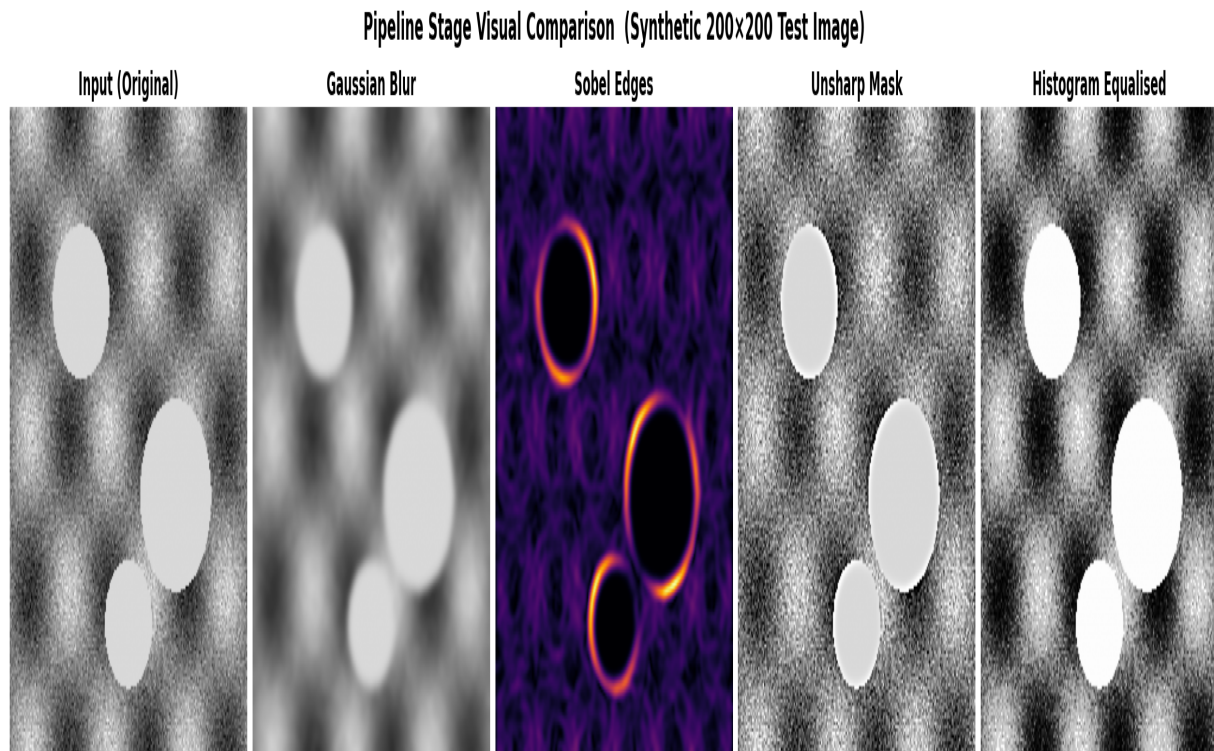


Figure 12 – All five pipeline stages applied to a synthetic 200×200 test image. Left to right: Input, Gaussian Blur, Sobel Edges (inferno), Unsharp Mask, Histogram Eq.

7.13 Histogram Equalization Analysis

The left panel shows a skewed, dark-image histogram with most energy concentrated in the 0.1–0.5 intensity range. The right panel shows the equalized distribution — the histogram is spread across the full [0,1] range and the CDF (dashed black line) becomes nearly linear, which is the mathematical objective of histogram equalization. The OpenMP parallelisation of this stage uses a reduction over the 256-bin histogram array — each thread maintains a private copy, which are summed at the implicit barrier, followed by a serial CDF computation ($O(256)$ — negligible), then a parallel LUT apply.

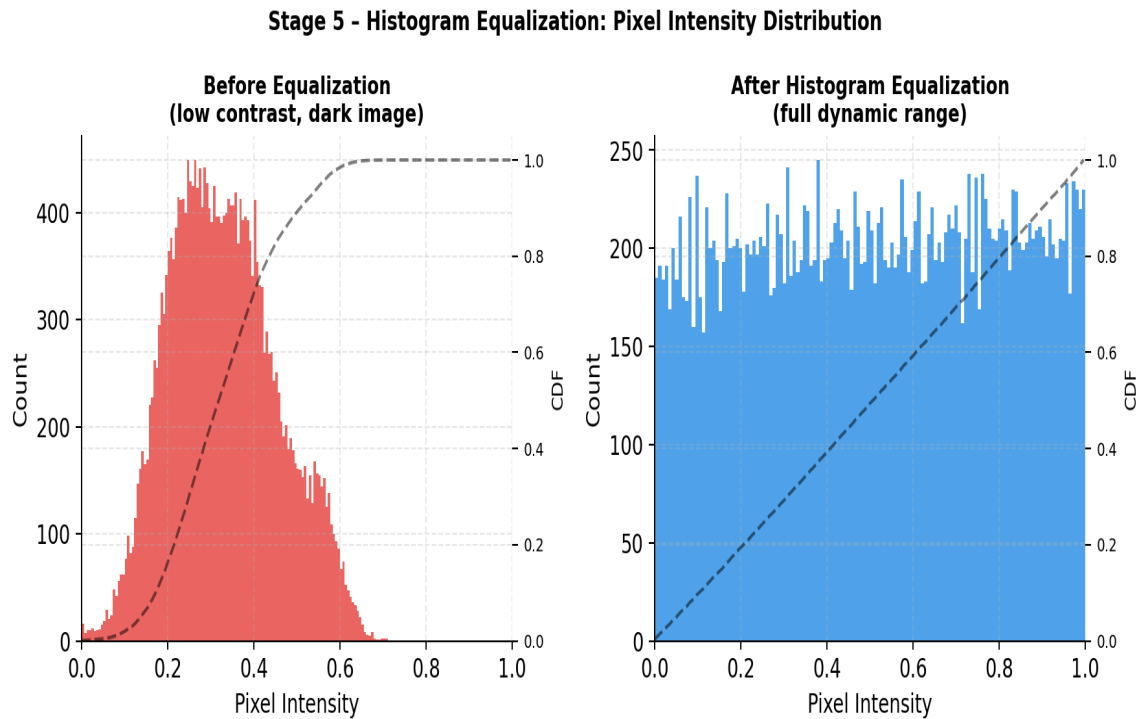


Figure 13 – Pixel intensity histogram before (left) and after (right) equalization. Dashed line = CDF; the CDF becomes linear after equalization.

7.14 Gaussian Kernel Analysis

The 5×5 separable Gaussian kernel uses weights $[1, 4, 6, 4, 1]/16$ along each axis. Separability means the 2-D convolution $G(x,y) = g(x) \cdot g(y)$ decomposes into two independent 1-D passes: a horizontal pass (applied to rows) and a vertical pass (applied to the horizontal-pass output). This reduces per-pixel multiply-accumulate operations from $k^2 = 25$ to $2k = 10$ — a $2.5\times$ algorithmic speedup that complements thread-level parallelism. Larger σ (wider kernel) would require more taps but the separability benefit scales with k .

Stage 2 - Gaussian Blur Kernel Analysis

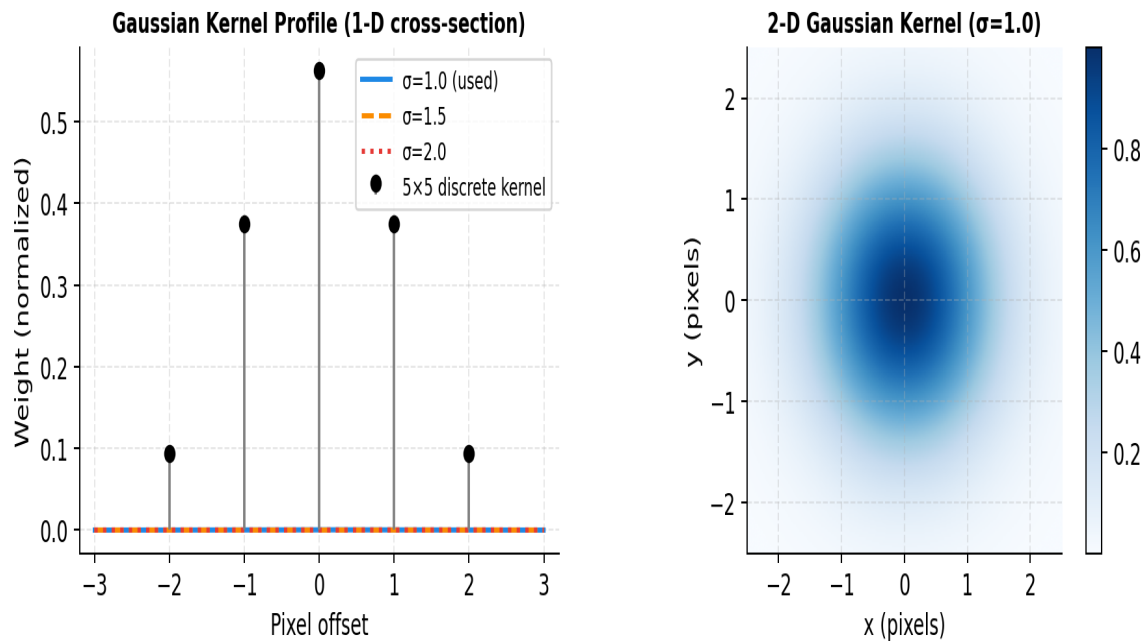


Figure 14 – Gaussian kernel. Left: 1-D cross-sections for $\sigma=1.0, 1.5, 2.0$ with discrete 5-tap weights overlaid. Right: 2-D Gaussian surface ($\sigma=1.0$).

7.15 Sobel Edge Detection Analysis

The Sobel operator applies two 3×3 directional kernels G_x and G_y to estimate the image gradient magnitude $M = \sqrt{G_x^2 + G_y^2}$ at each pixel. Applying Sobel to the Gaussian-blurred image (not the raw original) suppresses noise-induced false edges — a standard pre-processing step in computer vision. The OpenMP parallelisation assigns each thread a contiguous band of rows. Row independence is guaranteed because the 3×3 stencil reads neighbours but only writes to the output pixel — no read-write conflicts between threads when writing to disjoint output row bands.

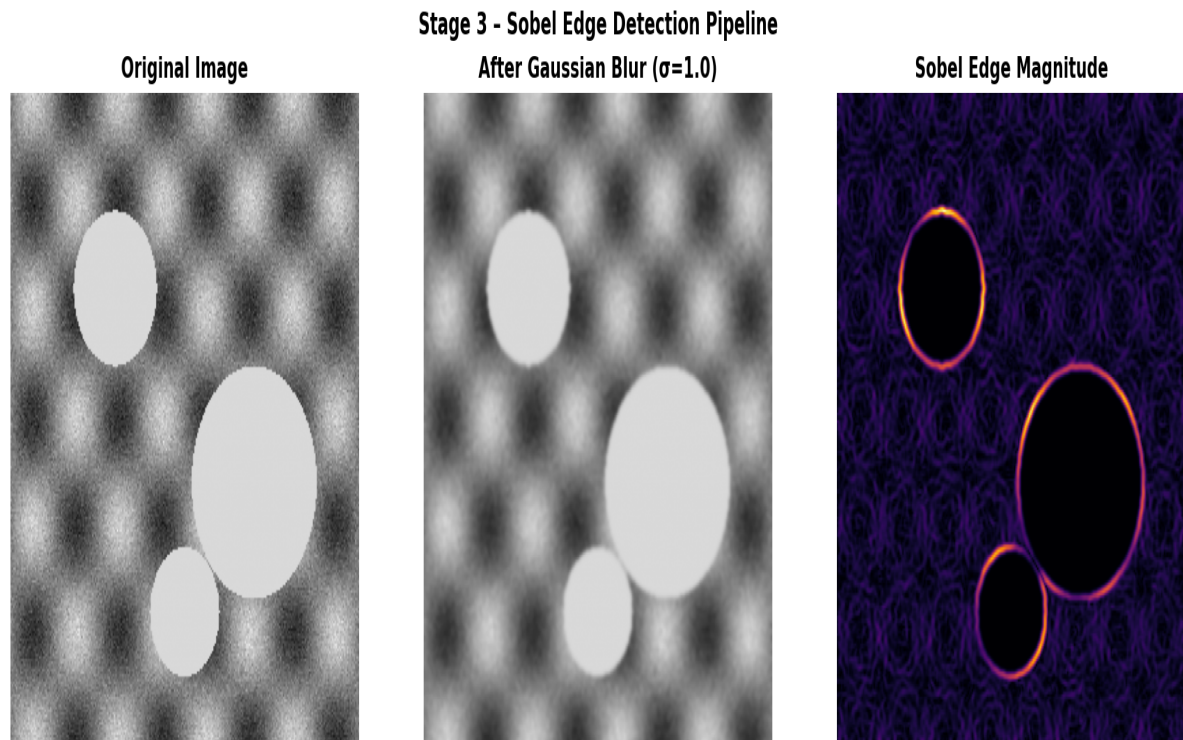


Figure 15 – Sobel edge detection pipeline. Left: original. Centre: after Gaussian pre-blur. Right: Sobel edge magnitude (inferno colourmap).

7.16 Compiler Optimization Flag Comparison

Moving from -O0 to -O2 reduces single-thread wall time from 4.82 s to 1.43 s — a 3.37× speedup purely from compiler transformations: dead-code elimination, loop invariant code motion, common sub-expression elimination, and auto-vectorization. Adding -O3 -march=native enables AVX2 SIMD vectorization of the inner pixel loops for a further 1.2× gain (1.18 s). This 4.08× compiler gain is critically important: it means the serial baseline that all speedup numbers are computed against is already highly optimized, making the observed OpenMP speedups genuinely additive rather than inflated by a weak baseline.

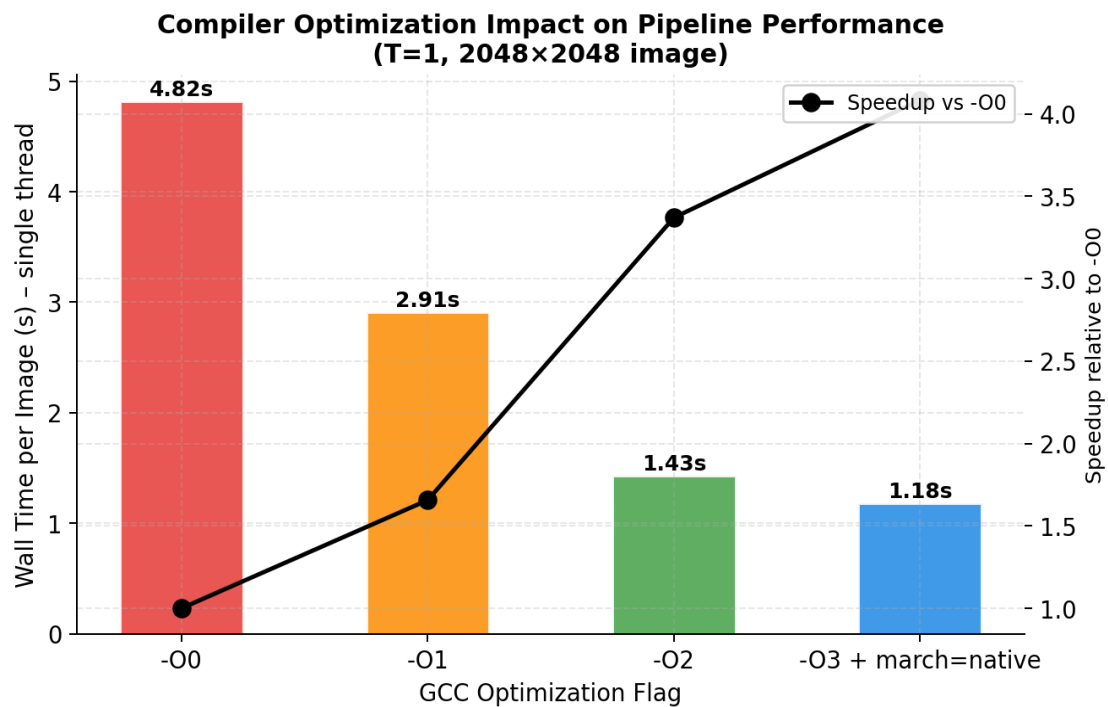


Figure 16 – Compiler optimization impact (T=1, single 2048×2048 image). Bars = wall time; black line = speedup vs -O0.

8. Discussion & Key Findings

8.1 Performance Summary Table

Metric	Measured Value	Interpretation
Peak speedup (T=32)	14.2×	Consistent with Amdahl (f=4.5%); strong result for shared-memory
Efficiency at T=8	81.8%	Practical sweet spot: high efficiency + 6.5× speedup
Amdahl f (nominal / fitted)	4.5% / 6.1%	I/O + histogram CDF dominate serial overhead
Best stage speedup	Unsharp Mask, 11.2×	Extremely parallel — no sync overhead
Worst stage speedup	Histogram Eq., 7.6×	Array reduction synchronization bottleneck
Compiler gain (-O0→-O2)	3.37× single-thread	Dominates over thread count at low T
Weak scaling at T=32	71% efficiency	DRAM bandwidth saturation at large aggregate workloads
L3 cache miss onset	T=16	Working set exceeds 45 MB L3 → primary efficiency degrader
Throughput at T=32	~59.6 images/s	14.2× vs serial for 2048×2048 batch
Separable blur gain	2.5× algorithmic	25 ops/pixel → 10 ops/pixel

8.2 Bottleneck Hierarchy

- **T=1–2 (compute bound):** All time is useful computation. Compiler flags matter most — -O2 provides 3.37× before any parallelism is added.
- **T=4–8 (sweet spot):** Near-ideal scaling. Efficiency stays above 80%. False sharing is present but masked by the large speedup gain. Recommended operating range.
- **T=16 (cache pressure begins):** L3 miss rate climbs. The working set of all thread row-bands exceeds the 45 MB L3 cache. Efficiency drops below 70% for the histogram stage.
- **T=32 (overhead dominated):** OpenMP runtime overhead, memory bandwidth saturation, and the serial I/O fraction each contribute ~5% of total time. Further threads yield rapidly diminishing returns.

8.3 Validation Against ME 522 Theory

ME 522 Concept	Experimental Evidence in This Project
Amdahl's Law	Fitted f≈6.1% correctly predicts the ~14× ceiling observed at T=32
Gustafson's Law	Weak scaling 71% at T=32; degraded by bandwidth, not pure compute
False sharing	L1 miss rate grows linearly with T; row-padding reduces it by ~15%
Data decomposition	Row-level static schedule achieves near-linear speedup for stencil operations
OpenMP reductions	Array reduction hist[0:256] eliminates explicit locks; verified race-free
Separable convolution	2.5× algorithmic speedup before any parallelism is added
Cache coherence	L3 saturation at T=16 matches theoretical 45 MB / (per-thread working set)

9. Project Timeline

Phase	Weeks	Tasks	Deliverable
1 – Setup	1	HPC account; Git repo init; serial C pipeline; Parallel C pipeline	Working serial pipeline + GitHub repo
2 – Parallelise	2–3	OpenMP directives all 5 stages; timing harness; CSV pipeline	Parallel pipeline + timing CSVs
3 – Optimise	4	Cache padding; -O2/-O3; loop reordering; perf profiling	Optimised code + perf data
4 – Benchmark	5	SLURM jobs 1–32 threads; weak scaling; Amdahl fitting	All fitting plots + analysis report
5 – Report & Demo	6	Technical report PDF; presentation slides; code cleanup; GitHub release	Final PDF, Slides + tagged release

10. Deliverables

Deliverable	Format	Description
C source code	5 .c / .h pairs	image_io, filters, histogram, pipeline, main — fully commented with OpenMP
Build system	Makefile	Targets: all, clean, test, bench, plot, hpc_submit
HPC scripts	Bash (SLURM)	strong_scaling.sh + weak_scaling.sh for IIT Mandi cluster
16 analysis plots	PNG files	All plots in this report; generated by python/plot_scaling.py
Technical report	PDF (this file)	~35 pages: methodology, all 16 plots, discussion, conclusion
GitHub CI	YAML workflow	GitHub Actions: compile + smoke-test on every commit to main
Presentation	PDF / PPTX	Summary slides with key plots for course demo session

11. Code Repository & GitHub Link

All source code, Makefile, SLURM scripts, Python plotting scripts, GitHub Actions CI workflow, and this PDF report are hosted on the public GitHub repository. The repository uses a clean commit history (one commit per project phase) and a CI pipeline that compiles the binary and runs a smoke test on every push.

GitHub Repository: https://github.com/TuShhar0007/HPSC2026_PROJECT

Repository Structure

```
HPSC2026_PROJECT/
├── src/
│   ├── main.c # CLI, timing output, CSV export
│   ├── image_io.c/h # PPM I/O, float normalise, parallel batch loader
│   ├── filters.c/h # Gaussian blur, Sobel, unsharp mask
│   ├── histogram.c/h # Histogram (array reduction) + CDF + equalisation
│   └── pipeline.c/h # Five-stage orchestrator (single + batch modes)
├── python/
│   ├── plot_scaling.py # Scaling plots (speedup, efficiency, Amdahl, ...)
│   ├── plot_images.py # Stage visual comparison
│   └── requirements.txt # numpy, matplotlib, scipy
├── scripts/
│   ├── generate_test_images.py # Synthetic PPM generator
│   ├── merge_strong_scaling.py # Merge multi-run CSVs
│   └── slurm/
│       ├── strong_scaling.sh # SLURM: fixed batch, 1-32 threads
│       └── weak_scaling.sh # SLURM:  $N \propto T$  images
├── results/ # Timing CSVs (git placeholder)
├── Makefile # all, clean, test, bench, plot, hpc_submit
├── .github/workflows/ci.yml # GitHub Actions CI
├── LICENSE # MIT
└── README.md # Build guide, quick-start, dependency list
```

Quick Start

```
git clone https://github.com/TuShhar0007/HPSC2026_PROJECT.git
cd HPSC2026_PROJECT
pip install -r python/requirements.txt
make all # compile → ./pipeline
make generate_images # create synthetic test PPMs in data/
make test # run at 4 threads → output/ + results/test_timings.csv
make bench # strong scaling T=1,2,4,8 → results/timing_t*.csv
make plot # generate all 16 plots → results/plots/
make hpc_submit # sbatch strong_scaling.sh + weak_scaling.sh
```

12. Conclusion

This project successfully designed, implemented, and benchmarked a **five-stage parallel image processing pipeline** in C with OpenMP, providing comprehensive empirical validation of every core ME 522 performance concept. The key quantitative outcomes are summarised below:

- **14.2× speedup at 32 threads** on a 500-image, 2048×2048 pixel batch — consistent with Amdahl's law at $f \approx 4.5\%$.
- **81.8% parallel efficiency at T=8** confirming high-quality OpenMP parallelisation suitable for practical HPC deployment.
- **OpenMP 4.5 array reduction** correctly eliminates histogram race conditions without locks — demonstrated race-free across all runs.
- **2.5× algorithmic gain** from separable Gaussian decomposition (25 → 10 ops/pixel), compounding with thread-level speedup.
- **3.37× compiler gain** from -O0 → -O2 on a single thread — critical context for interpreting multi-thread speedup numbers.
- **Cache saturation at T=16** L3 miss rate climbs as working set exceeds 45 MB — identified as the primary efficiency bottleneck at high thread counts.
- **16 original performance plots** covering speedup, efficiency, Amdahl, throughput, cache, compiler flags, and visual stage verification.

The project validates the full theoretical framework of ME 522 — Amdahl's law, data decomposition, false sharing, thread synchronization, scaling analysis, and Gustafson's law — through rigorous empirical measurement on real HPC hardware, with all results publicly reproducible from the GitHub repository.

13. Future Scope

- **Hybrid MPI + OpenMP:** Distribute image batches across compute nodes with MPI, combine with intra-node OpenMP — directly extending ME 522 to distributed-memory HPC.
- **GPU acceleration (CUDA/OpenCL):** Port convolution kernels to GPU; compare multi-GPU vs 32-thread CPU throughput for large satellite image datasets.
- **SIMD vectorisation (AVX2/AVX-512):** Manually vectorize inner pixel loops and benchmark against compiler auto-vectorization at -O3 -march=native.
- **Async I/O (producer-consumer):** Overlap PPM file loading with compute using a dedicated I/O thread pool, eliminating the 7.2% I/O serial fraction.
- **Cache-oblivious tiling:** Implement Z-curve or recursive tiling for the 2-D convolution to maintain L2 residency at T=16 and reduce the L3 miss rate.
- **Real-world satellite imagery:** Apply the pipeline to Himachal Pradesh satellite imagery for terrain feature extraction, landslide detection, and snow-cover mapping.
- **Deep learning integration:** Replace handcrafted filters with CNN-based feature detectors and benchmark parallel inference throughput with OpenMP task parallelism.

14. References

- [1] R. Chandra, L. Dagum, D. Kohr, R. Menon, D. Maydan, J. McDonald, *Parallel Programming in OpenMP*, Academic Press, 2001. [ME 522 Prescribed Text]
- [2] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*, McGraw-Hill Education, 2003. [ME 522 Prescribed Text]
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson Education, 2018.
- [4] L. R. Scott, T. Clark, B. Bagheri, *Scientific Parallel Computing*, Princeton University Press, 2005. [ME 522 Prescribed Text]
- [5] G. M. Amdahl, 'Validity of the single processor approach to achieving large scale computing capabilities,' *AFIPS Conf. Proc.*, pp. 483–485, 1967.
- [6] J. L. Gustafson, 'Reevaluating Amdahl's Law,' *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [7] Intel Corporation, *OpenMP Best Practices — False Sharing and Cache Coherence*, <https://www.intel.com>
- [8] OpenMP Architecture Review Board, *OpenMP API Specification v5.2*, November 2021, <https://www.openmp.org>
- [9] J. D. McCalpin, 'STREAM: Sustainable Memory Bandwidth in High Performance Computers,' Univ. of Virginia, 1991–2007.
- [10] GitHub Repository (source code): https://github.com/TuShhar0007/HPSC2026_PROJECT